SOFTWARE ENGINEERING · CHAPTER 02

Spring Framework

A modular Java framework that simplifies application development through dependency injection, inversion of control, declarative configuration and reusable infrastructure: from build tools and JDBC to Spring Boot web applications.

Spring Framework 7.0 **Spring Boot 4.1** **JDK 25 LTS** **Tomcat 11 · Servlet 6.1 · JSP 4.0** **Maven 3.9 · Gradle 9.7**

AGENDA

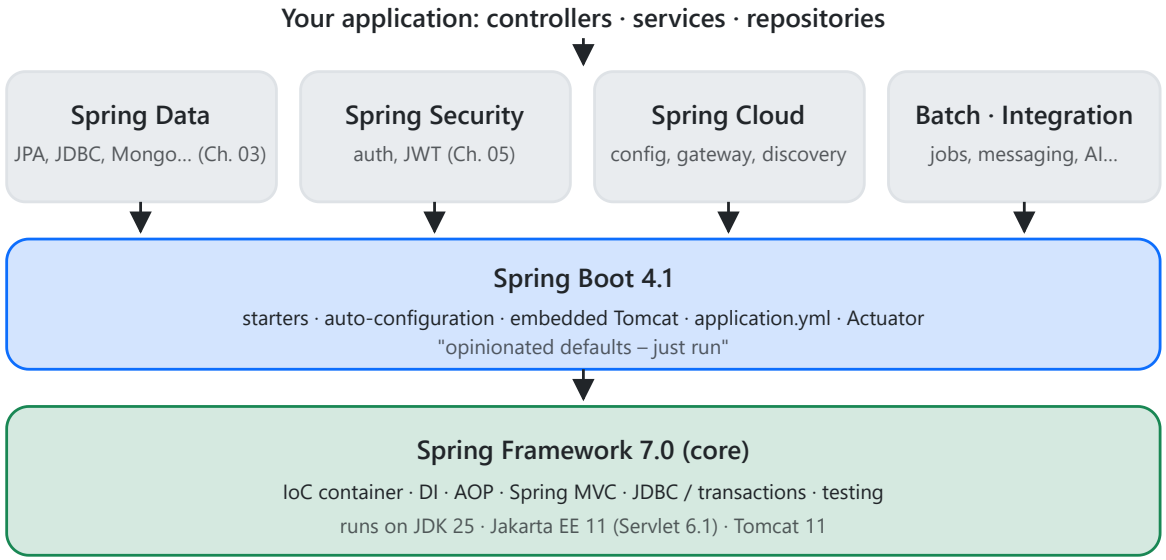
What we will cover

1. Introduction: the Spring ecosystem
2. Build tools (Maven, Gradle)
3. JavaServer Pages (JSP)
4. JDBC
5. Inversion of control and dependency injection
6. Bean definitions, scopes, and lifecycle
7. Component scanning and configuration classes
8. Spring Boot and auto-configuration
9. Application properties and profiles
10. Spring MVC controllers and request mappings
11. Validation, data binding, and exception handling
12. Spring transactions and declarative aspects
13. Actuator, logging, and externalized configuration
14. Layered architecture and Spring best practices
15. Chapter Quiz
16. Lab-02

Click any item to jump directly to that topic.

What is Spring, and why does everyone use it?

- **Spring Framework** (2003, Rod Johnson) started as a lightweight alternative to heavyweight J2EE: plain Java objects (POJOs) wired together by a *container*.
- Its core idea, **Inversion of Control**, means the framework creates and connects your objects; you only declare what you need.
- Around the core grew an **ecosystem**: Spring Boot (auto-configuration, embedded server), Spring Data, Spring Security, Spring Cloud…
- **Spring Boot 4.1** builds on **Spring Framework 7.0**, requires **JDK 17+** (we use 25) and Jakarta EE 11 APIs.
- Used by the majority of Java back-ends: microservices, REST APIs, batch jobs, web apps.



TOPIC 1

Build tools: Maven and Gradle

You met Maven in Chapter 01. Gradle solves the same problem with a Kotlin/Groovy *script* instead of XML, incremental builds and a build cache. Spring supports both equally; Spring Initializr generates either.

```
// build.gradle.kts – Gradle 9.7 with the Kotlin DSL
plugins {
    java
    id("org.springframework.boot") version "4.1.1"
    id("io.spring.dependency-management") version "1.1.7"
}

group = "edu.itc.se"
version = "1.0"

java { toolchain { languageVersion = JavaLanguageVersion.of(25) } }

repositories { mavenCentral() }

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-webmvc")
    implementation("org.springframework.boot:spring-boot-starter-validation")
    runtimeOnly("com.h2database:h2")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}

tasks.test { useJUnitPlatform() }
```

	Maven 3.9	**Gradle 9.7**
Descriptor	`pom.xml` (declarative XML)	`build.gradle(.kts)` (script)
Model	Fixed lifecycle phases	Task graph, incremental, cached
Wrapper	`./mvnw` (pins Maven version)	`./gradlew` (pins Gradle version)
Build & test	`mvn verify`	`./gradlew build`
Run Boot app	`mvn spring-boot:run`	`./gradlew bootRun`
Executable jar	`mvn package` → `target/*.jar`	`./gradlew bootJar` → `build/libs/*.jar`
Dependency tree	`mvn dependency:tree`	`./gradlew dependencies`

Always commit the wrapper (`mvnw`/`gradlew` + `.mvn/` or `gradle/wrapper/`). Everyone, including CI, then builds with the exact same tool version without installing anything.

How Spring Boot tames dependencies: starters, parent POM, BOM

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- inherit versions + plugin config for ~200 libraries -->
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
  </parent>

  <groupId>edu.itc.se</groupId>
  <artifactId>lab02-spring</artifactId>
  <version>1.0</version>

  <properties><java.version>25</java.version></properties>

  <dependencies>
    <!-- no <version>: managed by the parent -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

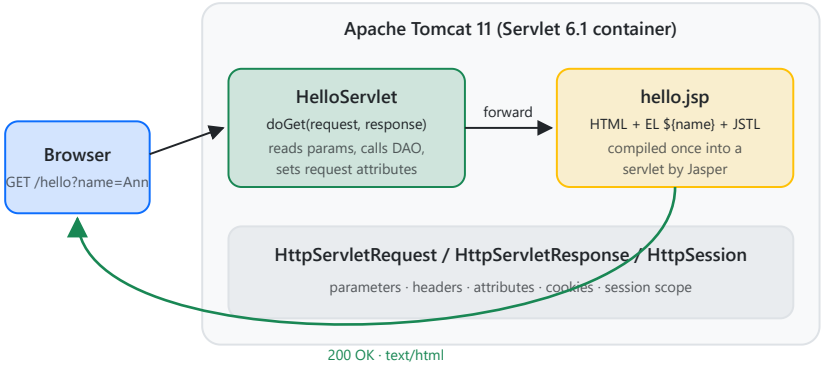
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>  <!-- builds the fat jar -->
  </plugin></plugins></build>
</project>
```

- **Starter** = one dependency that pulls in a coherent, tested set of libraries. `spring-boot-starter-webmvc` brings Spring MVC, embedded Tomcat, Jackson JSON, logging. (Before Boot 4 it was called `spring-boot-starter-web`.)
- **Parent POM / BOM** (Bill of Materials) fixes compatible versions for hundreds of artifacts, so you omit `<version>`. Never upgrade a transitive library on your own; upgrade Boot.
- **spring-boot-maven-plugin** repackages the jar as an *executable fat jar* containing all dependencies: `java -jar target/lab02-spring-1.0.jar`.

Starter	Gives you
`starter-webmvc`	REST & MVC, Tomcat, Jackson
`starter-validation`	Jakarta Bean Validation (Hibernate Validator)
`starter-jdbc`	`JdbcTemplate`/`JdbcClient`, HikariCP pool, transactions
`starter-data-jpa`	Hibernate + Spring Data repositories (Ch. 03)
`starter-actuator`	Health, metrics, env endpoints
`starter-aop`	AspectJ-style aspects
`starter-test`	JUnit 6, AssertJ, Mockito, MockMvc

🚀 start.spring.io (Spring Initializr) generates this skeleton for Maven or Gradle in seconds; IntelliJ and VS Code embed it.

Before Spring MVC: Servlets and JavaServer Pages



- A **Servlet** is a Java class that handles HTTP requests inside a *container* (Tomcat). One instance serves many requests on many threads (Chapter 01 in practice!).
- A **JSP** is an HTML page with embedded expressions; the container compiles it into a servlet on first use. JSP 4.0 ships with Tomcat 11.
- **Model 2 (MVC)**: the servlet is the *controller*, plain objects are the *model*, the JSP is the *view*. Spring MVC generalises exactly this pattern.
- Use the **Expression Language** `${…}` and **JSTL** tags; never write Java scriptlets `<% … %>` in pages.

```java
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    private final ProductDao dao = new ProductDao();      // no DI yet…

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        String name = req.getParameter("name");
        req.setAttribute("name", name == null ? "World" : name);
        req.setAttribute("products", dao.findAll());
        req.getRequestDispatcher("/WEB-INF/jsp/hello.jsp").forward(req, resp);
    }
}
```

Writing a JSP page: directives, EL and JSTL

```jsp
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c"   uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<!DOCTYPE html>
<html>
<head><title>Products</title></head>
<body>
  <h1>Hello, <c:out value="${name}"/>!</h1>   <%-- c:out escapes HTML --%>

  <c:if test="${empty products}">
    <p>No products yet.</p>
  </c:if>

  <table>
    <c:forEach var="p" items="${products}" varStatus="row">
      <tr class="${row.index % 2 == 0 ? 'even' : 'odd'}">
        <td>${p.id}</td>
        <td><c:out value="${p.name}"/></td>
        <td><fmt:formatNumber value="${p.price}" type="currency"/></td>
      </tr>
    </c:forEach>
  </table>

  <form method="post" action="${pageContext.request.contextPath}/products">
    <input name="name"> <input name="price" type="number" step="0.01">
    <button>Add</button>
  </form>
</body>
</html>
```

Element	Purpose
`<%@ page %>`, `<%@ taglib %>`	Directives: content type, imports, tag libraries
`${expr}`	EL: reads beans/attributes from page → request → session → application scope; `${p.name}` calls `getName()`
`<c:if>`, `<c:choose>`, `<c:forEach>`	JSTL core: conditionals and loops
`<c:out>`, `<c:url>`	HTML-escaped output, URL rewriting
`<fmt:formatNumber>`, `<fmt:formatDate>`	Locale-aware formatting
`<%-- --%>`	JSP comment (not sent to the browser)

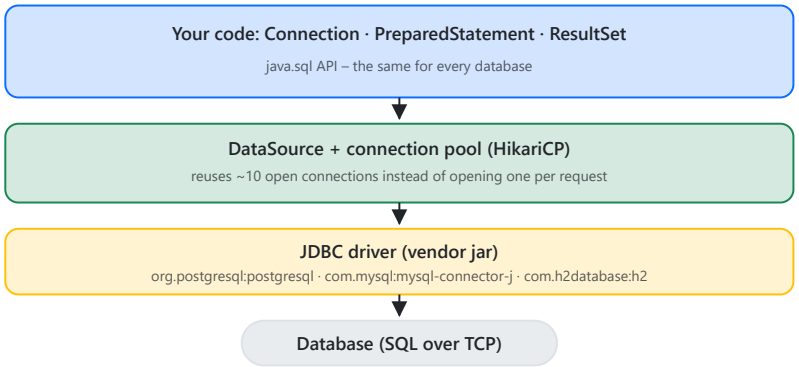
Using JSP inside a Spring Boot app

- Add `org.apache.tomcat.embed:tomcat-embed-jasper` and `org.glassfish.web:jakarta.servlet.jsp.jstl`.
- Put pages under `src/main/webapp/WEB-INF/jsp/` and package as `war` (JSPs cannot be served from a fat jar).
- Configure the view resolver in `application.yml`:

```yaml
spring:
  mvc:
    view:
      prefix: /WEB-INF/jsp/
      suffix: .jsp
```

⚠ Modern Spring projects usually prefer **Thymeleaf** (natural HTML templates, works from a jar). JSP is still widespread in existing systems, so you must be able to read and maintain it.

JDBC: Java Database Connectivity



- `jdbc:postgresql://localhost:5432/shop` – the URL selects the driver.
- Always use `PreparedStatement` with `?` placeholders: prevents **SQL injection** and lets the DB cache the plan.
- Connections, statements and result sets are resources: **try-with-resources**.

```
public List<Product> findCheaperThan(BigDecimal max) {
    String sql = "SELECT id, name, price FROM product WHERE price < ?";
    List<Product> result = new ArrayList<>();

    // 1. borrow from pool
    try (Connection con = dataSource.getConnection();
         // 2. parse once
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setBigDecimal(1, max);  // 3. bind safely
        try (ResultSet rs = ps.executeQuery()) {  // 4. execute
            while (rs.next()) {  // 5. map rows
                result.add(new Product(rs.getLong("id"),
                                       rs.getString("name"),
                                       rs.getBigDecimal("price")));
            }
        }
    } catch (SQLException e) {  // checked exception
        throw new IllegalStateException("query failed: " + sql, e);
    }  // 6. auto-close all
    return result;
}

public long insert(Product p) throws SQLException {
    String sql = "INSERT INTO product(name, price) VALUES (?, ?)";
    try (Connection con = dataSource.getConnection();
         PreparedStatement ps = con.prepareStatement(sql,
                 Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, p.name());
        ps.setBigDecimal(2, p.price());
        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            keys.next();
            return keys.getLong(1);
        }
    }
}
```

Pain points: 20 lines for one query, checked `SQLException` everywhere, manual mapping, easy to leak connections, transactions by hand with `con.setAutoCommit(false)`. Spring fixes all of these on the next slide.

Spring JDBC: `JdbcClient` and `JdbcTemplate`

```java
@Repository  // Spring bean + exception translation
public class ProductRepository {
    // auto-configured by spring-boot-starter-jdbc
    private final JdbcClient jdbc;
    public ProductRepository(JdbcClient jdbc) { this.jdbc = jdbc; }

    public List<Product> findCheaperThan(BigDecimal max) {
        return jdbc.sql("SELECT id, name, price FROM product WHERE price < :max")
                   .param("max", max)
                   // maps columns → record components
                   .query(Product.class)
                   .list();
    }

    public Optional<Product> findById(long id) {
        return jdbc.sql("SELECT id, name, price FROM product WHERE id = :id")
                   .param("id", id)
                   .query(Product.class)
                   .optional();
    }

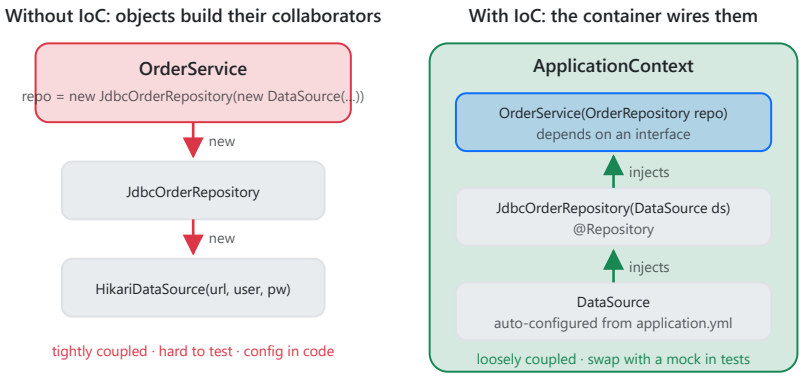
    public long insert(Product p) {
        var keys = new GeneratedKeyHolder();
        jdbc.sql("INSERT INTO product(name, price) VALUES (:name, :price)")
            .paramSource(p)  // binds record components by name
            .update(keys);
        return keys.getKey().longValue();
    }

    public int updatePrice(long id, BigDecimal price) {
        return jdbc.sql("UPDATE product SET price = :price WHERE id = :id")
                   .param("price", price).param("id", id)
                   .update();                // rows affected
    }
}
```

- **Template pattern**: Spring opens/closes the connection, creates the statement, iterates the result set and closes everything. You supply SQL, parameters and mapping.
- **Named parameters** `:max` instead of positional `?`.
- **Exception translation**: `SQLException` becomes an unchecked, vendor-independent `DataAccessException` hierarchy (`DuplicateKeyException`, `EmptyResultDataAccessException`…).
- `JdbcClient` (fluent, Spring 6.1+) sits on top of the classic `JdbcTemplate`; both are fine.
- Spring Boot creates the `DataSource` (HikariCP) from properties and runs `schema.sql` / `data.sql` at start-up for embedded DBs.

```yaml
# application.yml
spring:
  datasource:
    url: jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1
    username: sa
    password: ""
  sql:
    init:
      mode: always          # run schema.sql + data.sql
  h2:
    console:
      enabled: true         # http://localhost:8080/h2-console
```

Inversion of Control and Dependency Injection



- **Inversion of Control**: you don't call the framework, the framework calls you. The *container* (`ApplicationContext`) instantiates your classes and manages their lifecycle.
- **Dependency Injection** is the mechanism: the container passes collaborators into your object (constructor, setter or field) instead of the object looking them up.
- Depend on **interfaces**; the implementation is chosen by configuration, not by code.
- A managed object is called a **bean**.

```
public interface OrderRepository {
    Order save(Order o);
    Optional<Order> findById(long id);
}

@Service
public class OrderService {
    private final OrderRepository repo;  // final: set once, never null
    private final Clock clock;

    // Constructor injection – no @Autowired needed with a single constructor
    public OrderService(OrderRepository repo, Clock clock) {
        this.repo = repo;
        this.clock = clock;
    }

    public Order place(Order draft) {
        return repo.save(draft.withPlacedAt(Instant.now(clock)));
    }
}

// In a unit test: no Spring at all
var service = new OrderService(new InMemoryOrderRepository(),
                               Clock.fixed(Instant.EPOCH, ZoneOffset.UTC));
```

Three injection styles, one recommendation

```java
// 1) Constructor injection ✓ recommended
@Service
public class ReportService {
    private final OrderRepository orders;
    private final MailSender mail;
    public ReportService(OrderRepository orders, MailSender mail) {
        this.orders = Objects.requireNonNull(orders);
        this.mail = mail;
    }
}

// 2) Setter injection – for optional or reconfigurable dependencies
@Service
public class ExportService {
    // sensible default
    private Formatter formatter = new CsvFormatter();
    @Autowired(required = false)
    public void setFormatter(Formatter formatter) {
        this.formatter = formatter;
    }
}

// 3) Field injection ✗ avoid: hidden dependencies, not testable without reflection
@Service
public class LegacyService {
    @Autowired private OrderRepository orders;
}

// Choosing among several candidates
@Service
public class CheckoutService {
    public CheckoutService(@Qualifier("stripe") PaymentGateway gateway,
                           // all beans of that type
                           List<Validator> validators,
                           Optional<AuditLog> audit,  // may be absent
                           // lazy / optional lookup
                           ObjectProvider<Cache> cache) {
        …
    }
}
```

Style	Pros	Cons
Constructor	Immutable, mandatory deps obvious, trivially unit-testable, detects circular deps	Many params reveal a class doing too much (good!)
Setter	Optional deps, can re-inject	Object may be half-configured
Field	Short	Hides deps, needs reflection, no `final`, encourages god-classes

Resolution rules

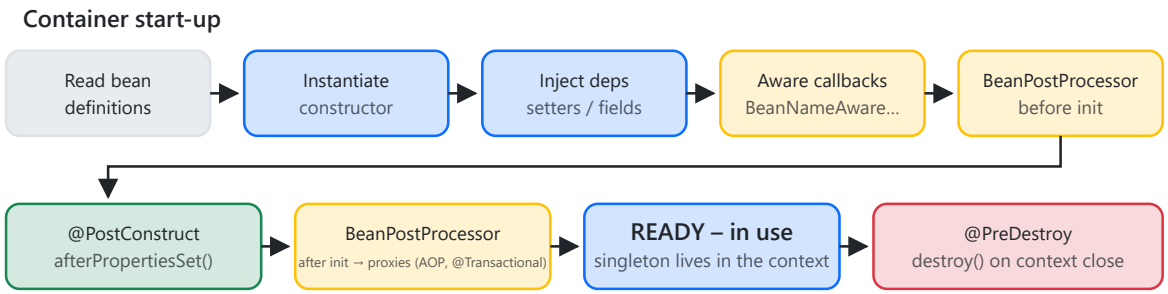
By **type** first. Exactly one candidate → injected.

Several candidates → `@Primary` wins, else `@Qualifier("name")`, else parameter name matching the bean name.

None → `NoSuchBeanDefinitionException` at start-up (fail fast, not at first use).

Circular constructor dependencies → start-up error; redesign (extract a third bean).

Bean definitions, scopes and lifecycle



Singletons are created eagerly at start-up (errors surface immediately); @Lazy defers creation to first use.

Scope	One instance per…
`singleton` (default)	container – must be **thread-safe** (stateless!)
`prototype`	`getBean()` / injection point; container does not destroy it
`request`	HTTP request (web apps)
`session`	HTTP session
`application`	`ServletContext`
`websocket`	WebSocket session

```
@Component
@Scope("prototype")
public class ReportBuilder { … }

@Component
@RequestScope  // request scope + proxy
public class RequestContext { … }

@Service
public class CacheWarmer {
    @PostConstruct void warmUp() { … }  // after injection
    @PreDestroy   void flush()  { … }  // on shutdown
}
```

Declaring beans: component scanning and `@Configuration`

Stereotype annotations on your own classes – found by *component scanning* of the package containing `@SpringBootApplication` and all sub-packages:

```
@Component   // generic managed bean
@Service     // business logic (semantic marker)
@Repository  // data access – adds DataAccessException translation
@Controller  // web layer (MVC views)
@RestController // @Controller + @ResponseBody (JSON)
@Configuration  // source of @Bean methods

// explicit name (default: "emailNotifierService")
@Service("emailNotifier")
public class EmailNotifierService implements Notifier { … }

@Repository
@Profile("!test")  // only when the 'test' profile is NOT active
public class JdbcOrderRepository implements OrderRepository { … }

@Repository
@Profile("test")
public class InMemoryOrderRepository implements OrderRepository { … }
```

Scanning is **package-based**. A class in `edu.itc.shop` is found by an application class in `edu.itc.shop`, not by one in `edu.itc.shop.app`. Keep the main class at the root package.

`@Bean` **methods** – for third-party classes you cannot annotate, or when construction needs logic:

```
@Configuration
public class AppConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();  // bean name = method name "clock"
    }

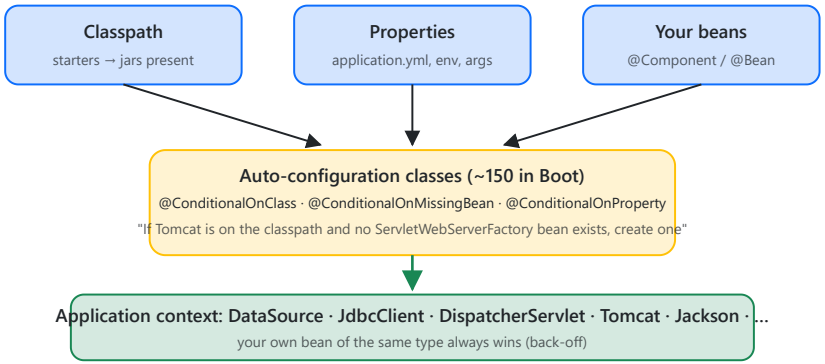
    @Bean
    @Primary  // preferred when several PaymentGateway beans exist
    // parameters are injected
    public PaymentGateway stripe(StripeProperties props) {
        return new StripeGateway(props.apiKey());
    }

    @Bean("paypal")
    @ConditionalOnProperty(name = "shop.paypal.enabled", havingValue = "true")
    public PaymentGateway paypal() {
        return new PayPalGateway();
    }

    @Bean(destroyMethod = "close")  // lifecycle hook for a library object
    public HttpClientPool httpPool() {
        return new HttpClientPool(20);
    }
}

// Composition & explicit scanning (rarely needed with Boot)
@Configuration
@Import({SecurityConfig.class, MessagingConfig.class})
@ComponentScan(basePackages = "edu.itc.shop.plugins")
public class RootConfig { }
```

Spring Boot: convention, auto-configuration, embedded server




```java
package edu.itc.shop;  // root package → scans edu.itc.shop.*

// = @Configuration + @EnableAutoConfiguration + @ComponentScan
@SpringBootApplication
public class ShopApplication {
    public static void main(String[] args) {
        // starts Tomcat on :8080
        SpringApplication.run(ShopApplication.class, args);
    }

    @Bean
    // runs once after start-up
    CommandLineRunner demo(ProductRepository repo) {
        return args -> repo.insert(
                new Product(null, "Keyboard", new BigDecimal("49.90")));
    }
}
```

```
$ mvn spring-boot:run  # dev: run from sources (DevTools restarts on change)
$ mvn package && java -jar target/shop-1.0.jar  # prod: one executable jar
$ java -jar shop.jar --server.port=9090  # override any property
```

- **Embedded server**: Tomcat runs inside your jar; no separate deployment, container-friendly.
- See what was decided: `--debug` flag prints the *condition evaluation report* (matched / unmatched auto-configs).
- Opt out: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

Application properties and profiles

```yaml
# src/main/resources/application.yml – defaults for every environment
spring:
  application:
    name: shop
server:
  port: 8080
shop:
  currency: EUR
  tax-rate: 0.20
  mail:
    from: noreply@shop.example
    retries: 3
logging:
  level:
    edu.itc.shop: INFO

---
# application-dev.yml  (activated with spring.profiles.active=dev)
spring:
  datasource:
    url: jdbc:h2:mem:shop
logging:
  level:
    edu.itc.shop: DEBUG
    org.springframework.jdbc.core: DEBUG

---
# application-prod.yml – secrets come from the environment, never from git
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
server:
  port: ${PORT:8080}       # default 8080 if PORT is unset
```

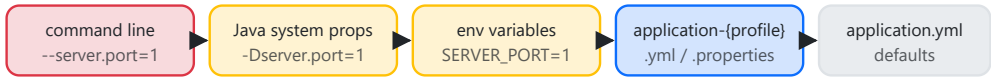
```java
// Type-safe binding – validated at start-up, IDE auto-completion
@ConfigurationProperties(prefix = "shop")
@Validated
public record ShopProperties(
        @NotBlank String currency,
        // tax-rate → taxRate
        @DecimalMin("0") @DecimalMax("1") BigDecimal taxRate,
        Mail mail) {
    public record Mail(@Email String from, @Min(0) int retries) {}
}

@SpringBootApplication
// or @EnableConfigurationProperties(ShopProperties.class)
@ConfigurationPropertiesScan
public class ShopApplication { … }

@Service
public class PricingService {
    private final ShopProperties props;
    public PricingService(ShopProperties props) { this.props = props; }
    BigDecimal gross(BigDecimal net) {
        return net.multiply(BigDecimal.ONE.add(props.taxRate()));
    }
}

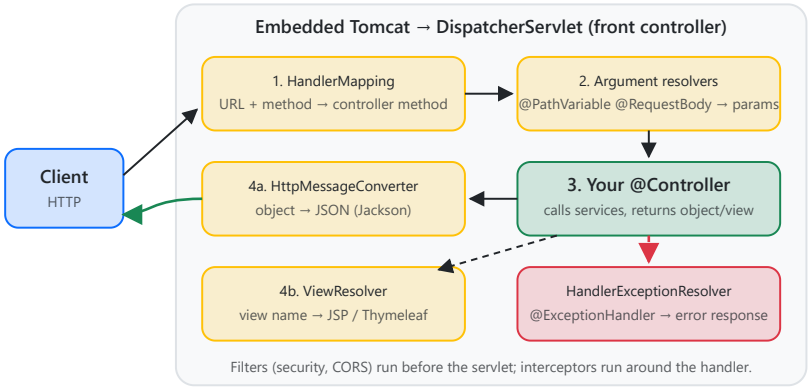
// Quick single values (fine for one-offs, no validation)
@Value("${server.port}") int port;
@Value("${shop.feature.beta:false}") boolean beta;
```

Precedence (highest wins)



Activate profiles: SPRING_PROFILES_ACTIVE=prod · --spring.profiles.active=dev,debug · @ActiveProfiles("test") in tests

Spring MVC: the `DispatcherServlet` and controllers




```java
@RestController  // every method returns a body, not a view
@RequestMapping("/api/products")                 // common prefix
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping  // GET /api/products?minPrice=10&page=0
    public List<ProductDto> list(
            @RequestParam(defaultValue = "0") BigDecimal minPrice,
            @RequestParam(defaultValue = "0") int page) {
        return service.find(minPrice, page);
    }

    @GetMapping("/{id}")                         // GET /api/products/42
    public ProductDto one(@PathVariable long id) {
        return service.get(id);  // ProductNotFoundException → 404
    }

    @PostMapping  // POST /api/products  {"name":"…","price":9.9}
    @ResponseStatus(HttpStatus.CREATED)
    public ProductDto create(@RequestBody @Valid CreateProduct cmd) {
        return service.create(cmd);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductDto> update(
            @PathVariable long id,
            @RequestBody @Valid CreateProduct cmd,
            @RequestHeader("If-Match") String etag) {
        return ResponseEntity.ok().eTag(etag).body(service.update(id, cmd));
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable long id) { service.delete(id); }
}
```

Request mappings and server-rendered views

```java
@Controller                                    // returns view names
@RequestMapping("/products")
public class ProductPageController {
    private final ProductService service;
    public ProductPageController(ProductService service) {
        this.service = service;
    }

    @GetMapping                                // GET /products
    public String list(Model model, @RequestParam(required = false) String q) {
        model.addAttribute("products", service.search(q));
        model.addAttribute("q", q);
        return "products/list";  // → /WEB-INF/jsp/products/list.jsp
    }

    @GetMapping("/new")
    public String form(Model model) {
        model.addAttribute("form", new ProductForm("", null));
        return "products/form";
    }

    @PostMapping  // form POST (application/x-www-form-urlencoded)
    public String create(@Valid @ModelAttribute("form") ProductForm form,
                         // must follow the validated param
                         BindingResult errors,
                         RedirectAttributes flash) {
        // re-render with messages
        if (errors.hasErrors()) return "products/form";
        long id = service.create(form.toCommand());
        flash.addFlashAttribute("message", "Product " + id + " created");
        // POST-redirect-GET pattern
        return "redirect:/products";
    }
}
```

Annotation	Binds
`@GetMapping("/x/{id}")` …	HTTP method + path; `consumes`/`produces` for content types
`@PathVariable`	Path segment `/products/42`
`@RequestParam`	Query string or form field `?q=usb`
`@RequestBody`	JSON/XML body → object (message converter)
`@ModelAttribute`	Form fields → object (data binding)
`@RequestHeader`, `@CookieValue`	Header / cookie value
`Model`, `ModelMap`	Data handed to the view
`ResponseEntity<T>`	Full control: status, headers, body

Return value → what happens

`String` in `@Controller` → view name, resolved by prefix/suffix; `"redirect:/path"` sends 302.

Any object in `@RestController` (or with `@ResponseBody`) → serialised by Jackson based on the `Accept` header.

`void` + `@ResponseStatus` → empty response with that status.

Static files in `src/main/resources/static/` (CSS, JS, images) are served automatically at `/`.

Validation, data binding and consistent error handling

```java
// DTO with Jakarta Bean Validation constraints (spring-boot-starter-validation)
public record CreateProduct(
        @NotBlank @Size(max = 80) String name,
        @NotNull @DecimalMin("0.01") @Digits(integer = 8, fraction = 2)
        BigDecimal price,
        @Pattern(regexp = "[A-Z]{3}") String currency,
        @Valid Dimensions dimensions) {  // cascade into nested object
    public record Dimensions(@Positive int widthMm, @Positive int heightMm) {}
}

// Custom constraint
@Target(FIELD) @Retention(RUNTIME) @Constraint(validatedBy = SkuValidator.class)
public @interface Sku {
    String message() default "invalid SKU";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// Business exception, mapped to a status
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(long id) {
        super("Product " + id + " not found");
    }
}

// Validating method parameters in services too
@Service @Validated
public class ProductService {
    public ProductDto get(@Positive long id) {
        return repo.findById(id).map(ProductDto::from)
                   .orElseThrow(() -> new ProductNotFoundException(id));
    }
}
```

```java
// One place for all error responses – RFC 9457 "problem details"
@RestControllerAdvice
public class ApiExceptionHandler {

    // @Valid failed
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalidBody(MethodArgumentNotValidException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST,
                                                  "Validation failed");
        pd.setTitle("Invalid request");
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
                .map(f -> Map.of("field", f.getField(),
                                 "message", f.getDefaultMessage()))
                .toList());
        return pd;
    }

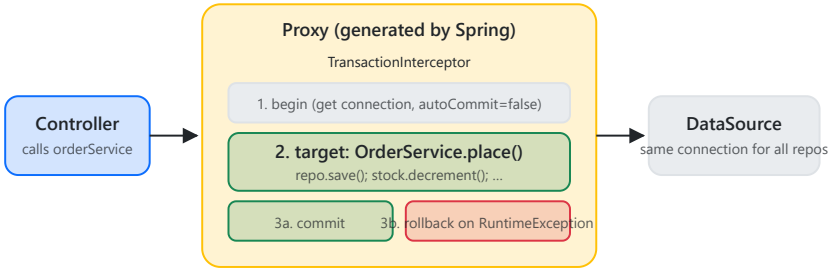
    @ExceptionHandler(ProductNotFoundException.class)
    ProblemDetail notFound(ProductNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
                                                ex.getMessage());
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    ProblemDetail conflict(DataIntegrityViolationException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT,
                                                "Constraint violated");
    }
}
```

```http
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
{
  "type": "about:blank",
  "title": "Invalid request",
  "status": 400,
  "detail": "Validation failed",
  "instance": "/api/products",
  "errors": [
    { "field": "price", "message": "must be greater than or equal to 0.01" }
  ]
}
```

- **Data binding** converts strings (query, form, path) to typed parameters; register a `Converter` or use `@DateTimeFormat` for custom formats.
- Enable `spring.mvc.problemdetails.enabled=true` for built-in ProblemDetail responses on common MVC errors.

Declarative transactions with `@Transactional`



Self-invocation (this.place() from another method of the same class) bypasses the proxy → no transaction!

Same mechanism for @Async, @Cacheable, @Retryable, security checks and your own aspects.

```java
@Service
public class OrderService {
    private final OrderRepository orders;
    private final StockRepository stock;
    public OrderService(OrderRepository orders, StockRepository stock) {
        this.orders = orders; this.stock = stock;
    }

    @Transactional                                  // all-or-nothing
    public Order place(PlaceOrder cmd) {
        Order order = orders.save(Order.from(cmd));
        for (var line : cmd.lines()) {
            int updated = stock.decrement(line.productId(), line.qty());
            // throwing a RuntimeException rolls the whole transaction back
            if (updated == 0) throw new OutOfStockException(line.productId());
        }
        return order;
    }

    @Transactional(readOnly = true)  // hint: no flush, may use read replica
    public List<Order> history(long customerId) {
        return orders.findByCustomer(customerId);
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW,
                   // roll back on checked exceptions too
                   rollbackFor = Exception.class,
                   timeout = 5,
                   isolation = Isolation.REPEATABLE_READ)
    public void audit(String event) { … }
}
```

Propagation	Behaviour
`REQUIRED` (default)	Join the caller's transaction or start one
`REQUIRES_NEW`	Suspend caller's, run in a fresh one (audit logs)
`SUPPORTS` / `NOT_SUPPORTED`	Run with or without whatever exists
`MANDATORY` / `NEVER`	Fail if there is no / is a transaction

Writing your own declarative aspect (Spring AOP)

```java
// spring-boot-starter-aop
@Aspect
@Component
public class TimingAspect {
    private static final Logger log =
            LoggerFactory.getLogger(TimingAspect.class);

    // Pointcut: every public method of every class in the service package
    @Pointcut("execution(public * edu.itc.shop.service..*(..))")
    void serviceMethods() {}

    @Around("serviceMethods()")  // advice wraps the join point
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long t0 = System.nanoTime();
        try {
            return pjp.proceed();  // call the real method
        } finally {
            log.info("{} took {} µs", pjp.getSignature().toShortString(),
                     (System.nanoTime() - t0) / 1_000);
        }
    }

    @AfterThrowing(pointcut = "@annotation(edu.itc.shop.Audited)",
                   throwing = "ex")
    public void audit(JoinPoint jp, Exception ex) {
        log.warn("audited method {} failed: {}",
                 jp.getSignature().getName(), ex.getMessage());
    }
}

// Custom annotation as a pointcut target
@Target(METHOD) @Retention(RUNTIME)
public @interface Audited {}
```

Term	Meaning
Cross-cutting concern	Logic needed in many places: logging, timing, security, transactions, caching
Aspect	Class bundling advice + pointcuts
Join point	A method execution on a bean
Pointcut	Expression selecting join points
Advice	`@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`
Weaving	Spring: runtime **proxies** (JDK dynamic or CGLIB)

Proxy limitations: only *public* methods called *from another bean* are advised. Self-calls, `private`/`final` methods and objects you `new` yourself are invisible to Spring AOP. Keep aspects thin and side-effect free.

Operating the app: Actuator, logging, externalized configuration

Actuator endpoint	Shows
`/actuator/health`	UP/DOWN + DB, disk, custom indicators; used by Kubernetes probes
`/actuator/info`	Build/git info, custom `info.*` properties
`/actuator/metrics`	JVM, HTTP, DataSource metrics (Micrometer → Prometheus)
`/actuator/env`	Effective properties and their sources
`/actuator/loggers`	View **and change** log levels at runtime (POST)
`/actuator/beans`, `/conditions`	All beans; auto-configuration decisions

```yaml
management:
  endpoints:
    web:
      exposure:
        # only health is exposed by default
        include: health,info,metrics,loggers
  endpoint:
    health:
      show-details: when-authorized
info:
  app:
    name: ${spring.application.name}
    version: @project.version@  # Maven resource filtering
```

```java
@Component
public class StockHealth implements HealthIndicator {   // custom check
    @Override public Health health() {
        int low = stock.countBelowMinimum();
        if (low == 0) return Health.up().build();
        return Health.down().withDetail("lowStockItems", low).build();
    }
}
```

```java
// Logging: code against SLF4J; Boot wires Logback underneath
private static final Logger log = LoggerFactory.getLogger(OrderService.class);

// placeholders {} instead of string concatenation
log.debug("placing order {} for customer {}", cmd.id(), cmd.customerId());
log.info("order {} placed, total={}", order.id(), order.total());
log.warn("stock low for product {}", productId);
// a Throwable as last argument prints the stack trace
log.error("payment failed for order {}", order.id(), ex);

// Correlate log lines of one request (MDC), e.g. in a Filter
MDC.put("requestId", UUID.randomUUID().toString());
```

```yaml
logging:
  level:
    root: INFO
    edu.itc.shop: DEBUG
    org.springframework.web: INFO
  pattern:
    console: "%d{HH:mm:ss.SSS} %-5level [%X{requestId}] %logger{36} - %msg%n"
  file:
    name: logs/shop.log  # rolling file in addition to console
```

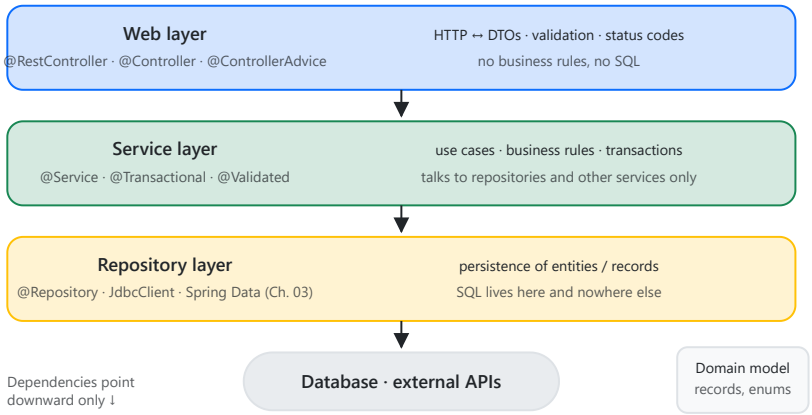
Externalized configuration = 12-factor

Same jar in every environment; behaviour changes only via **env variables / config files outside the jar** (`./config/application.yml` next to the jar is picked up automatically).

Secrets never in git: env vars, Docker/Kubernetes secrets, Vault or Spring Cloud Config.

Verify with `/actuator/env` which source supplied a value.

Layered architecture of a Spring application




```
edu.itc.shop
├── ShopApplication.java          @SpringBootApplication (root package)
├── config/                       AppConfig, ShopProperties, WebConfig
├── web/                          ProductController, ProductPageController, ApiExceptionHandler
│   └── dto/                      ProductDto, CreateProduct  (records, validated)
├── service/                      ProductService, OrderService, PricingService
├── repository/                   ProductRepository, OrderRepository (interfaces + Jdbc impls)
└── domain/                       Product, Order, OrderLine, Money  (records / entities)
```

- **DTOs at the edge**: never expose entities/records with DB shape directly; map to `ProductDto` so the API can evolve independently.
- **Thin controllers, rich services**: a controller method should read like "parse → call service → map response".
- **Transactions at the service boundary**, never in controllers or repositories.
- **Package by layer** (shown) is simplest for small apps; larger systems package *by feature* (`product/`, `order/`) to keep modules cohesive.
- Each layer is unit-testable in isolation because dependencies are injected interfaces.

TOPIC 13

Spring best practices

1 Constructor injection, final fields

Mandatory dependencies are explicit and immutable; classes stay testable with plain `new`.

2 Stateless singletons

Beans are shared across request threads. No mutable fields in services and controllers (see Chapter 01).

3 Let Boot manage versions

Use starters and the parent BOM; upgrade Boot as a whole. Commit the wrapper.

4 Typed configuration

`@ConfigurationProperties` records with validation instead of scattered `@Value`; profiles for environments; secrets from the environment.

5 Validate at the boundary

`@Valid` DTOs in controllers, one `@RestControllerAdvice`, ProblemDetail responses. Fail with the right status code.

6 Transactions in services

Short, at the use-case level, `readOnly` for queries. Remember the proxy: no self-invocation, public methods only.

7 Test the slices

Unit tests without Spring; `@WebMvcTest` + MockMvc for controllers; `@JdbcTest` for repositories; a few `@SpringBootTest` end-to-end (Chapter 06).

8 Observe in production

Actuator health/metrics, structured logs with request ids, sensible log levels per package.

9 Understand the magic

When something "just works", know which auto-configuration did it (`--debug`, `/actuator/conditions`). When it doesn't, read the start-up failure analyser message first.

Chapter quiz 10 questions

1. Which injection style does the Spring team recommend for mandatory dependencies?

- Field injection with @Autowired
- Setter injection
- Constructor injection
- Static factory lookup

2. What is the default scope of a Spring bean?

- prototype
- singleton
- request
- session

3. @SpringBootApplication is a combination of…

- @Controller + @Service + @Repository
- @Configuration + @EnableAutoConfiguration + @ComponentScan
- @Bean + @Import
- @RestController + @RequestMapping

4. server.port is 8080 in application.yml and 9090 in application-prod.yml. The app is started with --server.port=7070 and profile prod. Which port is used?

- 8080
- 9090
- 7070
- Start-up fails because of the conflict

5. A @Transactional public method is called from another method of the same class via this.method(). What happens?

- A transaction is started as usual
- No transaction: the call bypasses the proxy
- A nested transaction is created
- An exception is thrown

6. Which annotation binds a JSON request body to a method parameter?

- @RequestParam
- @ModelAttribute
- @RequestBody
- @PathVariable

7. A @Valid @RequestBody DTO fails validation in a @RestController. Without any custom handling the client receives…

- 500 Internal Server Error
- 400 Bad Request
- 422 Unprocessable Entity
- 200 with an empty body

8. Compared with raw JDBC, JdbcClient / JdbcTemplate do NOT do which of the following?

- Open and close connections
- Translate SQLException into DataAccessException
- Generate the SQL for you from the record type
- Bind named parameters

9. Which Actuator endpoint is typically used by Kubernetes liveness/readiness probes?

- /actuator/env
- /actuator/health
- /actuator/beans
- /actuator/loggers

10. What extra behaviour does @Repository add compared to @Component?

- Automatic transactions
- Translation of persistence exceptions into Spring's DataAccessException hierarchy
- Connection pooling
- JSON serialisation

WRAP-UP

Summary

Maven or Gradle + Spring Boot starters and the parent BOM manage the build and versions.

Servlets/JSP are the foundation; Spring MVC's `DispatcherServlet` generalises the Model-2 pattern.

JDBC is verbose; `JdbcClient` removes the boilerplate and translates exceptions.

IoC container + constructor injection = loosely coupled, testable beans (stateless singletons by default).

Beans come from component scanning or `@Bean` methods; Boot auto-configures the rest, your beans win.

Configuration is externalized: `application-{profile}.yml`, env vars, `@ConfigurationProperties`.

Controllers map HTTP to services; validate DTOs; one advice class returns ProblemDetail errors.

`@Transactional` and aspects work through proxies at the service layer.

Actuator, SLF4J logging and layered packages keep the app operable and maintainable.

Open Lab-02: 5 tasks + 1 challenge

Next chapter: 03 · Hibernate Framework and Spring Data JPA